AWS Elastic Beanstalk + Amplify Multi-Branch CI/CD Deployment Project

A Hands-On Debugging Journey & AWS Certified Developer – Associate (DVA-C02) Exam Preparation Report

---

Project: els-cicd (Elastic Beanstalk backend + Amplify frontend)

Repository: aws-engineering-journey / projects/els-cicd

Environment: AWS Elastic Beanstalk (Node.js 24, Amazon Linux 2023) — us-east-1

Exam Target: AWS Certified Developer – Associate (DVA-C02), Target Score: 900+

Report Date: July 4–5, 2026

This report documents a real, hands-on troubleshooting session while deploying a Node.js backend to Elastic Beanstalk with a multi-branch staging/production architecture. Every error encountered, its root cause, and the underlying AWS concept it teaches has been captured — because debugging real infrastructure failures builds far deeper exam-ready intuition than passively reading documentation.

Table of Contents

1. Project Architecture Overview

The project implements a multi-branch CI/CD pattern combining two AWS compute/hosting services:

```
GitHub Repo: aws-engineering-journey/projects/els-cicd
+-- backend/ -> AWS Elastic Beanstalk (Node.js API)
| +-- app.js
| +-- package.json
| +-- .ebextensions/
| +-- 01-environment.config
| +-- 02-nginx.config
+-- front-end/ -> AWS Amplify (static/React frontend)

Branch strategy:
main -> Production environment (myapp-prod)
dev -> Staging environment (myapp-staging)
```

Elastic Beanstalk handles the compute layer (EC2 + Auto Scaling + Load Balancer + Nginx reverse proxy), while Amplify independently handles frontend build/hosting with its own native branch-based CI/CD, removing the need for a separate CodePipeline for the frontend.

2. Timeline of the Debugging Journey

The table below chronicles every issue hit during implementation, in the order encountered. This is the most valuable part of the project — each row represents a real production-style failure mode.

#	Symptom	Root Cause	Fix
1	EB served the AWS Sample App instead [illegible]	app.js/package.json were placed inside .ebextensions/ by mistake, so eb deploy (git-based) found [illegible]	Moved app.js & package.json to backend/ root; kept only [illegible]
2	eb: command not found	AWS EB CLI was never installed	pip install awsebcli --upgrade inside the active venv
3	ValidationError: VersionLabel is None	Direct symptom of issue #1 — no committed source [illegible]	Resolved automatically once [illegible] committed to git
4	Deployed to us-east-1 instead of ap-south-1	aws configure default region and interactive eb init prompts [illegible]	Verified via .elasticbeanstalk/config.yml; re-ran eb init explicitly [illegible]
5	Duplicate environments across two regions	A failed eb terminate (name mismatch) was followed by a fresh eb init/eb create [illegible]	Manually [illegible]
6	InvalidParameterValueError: Must be Ready	Chained commands (eb create + eb deploy) were fired before the environment finished its state transition	Learned to poll eb status until Status: Ready before issuing the [illegible]
7	eb ssh failed: SSH not set up / wrong keypair	eb ssh --setup was run selecting keypairs (baston-host, [illegible]) whose private keys did not exist locally	Generated a new [illegible] ~/.ssh/ [illegible] select [illegible]
8	Instance deployment failed: Engine execution has encountered an error	Deploy fails ~5 seconds into 'Deploying new version' every time [illegible]	Iterative — isolated; confirmed package.json had express listed [illegible]
9	Environment health shows Green/Ok but wrong app runs	EB's health model considers the environment healthy once [illegible]	Learned to check Deployed Version matches expectation [illegible]

3. Root Cause Analysis: Why the Deploy Kept Failing

At the time of writing, the core mystery — the 'Engine execution has encountered an error' failure occurring exactly ~5 seconds after 'Deploying new version to instance(s)' — remained unresolved pending the full eb-engine.log contents. However, the diagnostic process itself is the real lesson here. The systematic elimination approach used was:

- **Rule out the application code:** app.js was reviewed and confirmed syntactically correct with no missing logic.
- **Rule out missing dependencies:** package.json was inspected and confirmed to declare express as a dependency, with a valid start script.
- **Rule out a specific .ebextensions directive:** the manually-scripted container_command running 'npm install --production' a second time was disabled as an isolation test, since Node.js platforms on Amazon Linux 2023 already run dependency installation automatically as part of the platform hook lifecycle.
- **Rule out CLI-vs-console log access issues:** eb logs and eb ssh both failed for infrastructure reasons (stale environment state, missing local SSH keys) unrelated to the deploy failure itself, so the console's Request Logs -> Full Logs -> Download flow was used as the reliable fallback.

Exam insight: this mirrors exactly how AWS expects a Developer Associate to triage a failed deployment — check application code, check dependency manifests, check .ebextensions/.platform customizations, then pull raw engine logs. Scenario-based exam questions frequently describe a similar chain of symptoms and ask you to identify the most likely next diagnostic step.

4. Key AWS Concepts Learned (Mapped to DVA-C02 Domains)

Domain	Concept Reinforced by This Project
Domain 1: Development with AWS Services (32%)	Elastic Beanstalk application versions, environment variables via --envvars, the Node.js platform's built-in np
Domain 2: Security (26%)	IAM instance profiles for EB environments; the difference between plaintext .ebextensions environment vari
Domain 3: Deployment (24%)	.ebextensions config precedence and alphabetical execution order; container_commands vs. platform hook
Domain 4: Troubleshooting and Optimization (18%)	Reading eb health, eb status, and eb logs output; understanding that environment health reflects version-ma

5. Critical Mistakes & How to Avoid Them

■ Placing application code inside .ebextensions/

.ebextensions/ is reserved exclusively for .config customization files (YAML/JSON) that configure the environment. Application source code must sit at the root of the deployed directory. Mixing the two causes eb deploy to interpret the directory as having no real source.

■ Chaining eb create and eb deploy without waiting

EB environment operations are asynchronous and state-gated. Always poll eb status until Status: Ready before issuing the next command. Running commands back-to-back in a single terminal block is a common cause of Must be Ready errors.

■ Assuming green health means the correct app is running

EB health checks validate that the environment is stable and serving the version it was told to serve — even if that version is a rolled-back fallback like the Sample Application. Always cross-check the Deployed Version field, not just the health color.

■ Selecting an EC2 keypair name without a matching local private key

eb ssh --setup lets you pick from any keypair name registered in AWS, but SSH will only work if the corresponding private key file exists in ~/.ssh/ on the machine running the command. Generate a fresh keypair locally (ssh-keygen) if unsure, and select that one.

■ Letting local .elasticbeanstalk/config.yml drift from actual deployed infrastructure

This file is local, per-directory state binding your CLI to a specific application/environment/region. A failed eb terminate followed by a new eb init can silently create a second, orphaned environment while your CLI context quietly points elsewhere. Always cat .elasticbeanstalk/config.yml when confused about which environment a command is targeting.

6. Elastic Beanstalk Deployment Policies Reference

A high-yield DVA-C02 topic. Know when to use each policy and its downtime/cost tradeoff.

Policy	Downtime?	Rollback Speed	Best For
All at once	Yes	Manual redeploy	Dev/test environments; fastest, cheapest, riskiest
Rolling	No (reduced capacity)	Manual, batch by batch	Cost-conscious prod with tolerance for temporarily reduced c
Rolling with additional batch	No	Manual, batch by batch	Prod environments needing full capacity maintained during d
Immutable	No	Fast (terminate new ASG)	Production deploys needing safe, fully-isolated testing of the
Traffic splitting (canary)	No	Fast (shift traffic back)	Gradual rollout with real-user validation before full cutover

The zero-downtime blue/green pattern used in this project's later stages (eb swap-environment-cnames) is functionally similar to Immutable/Canary in intent: run the new version in full isolation, validate, then cut over traffic instantly by swapping CNAMEs at the DNS/routing layer rather than replacing instances in place.

7. Command Reference Sheet

```
# Setup
pip install awsebcli --upgrade
aws configure
eb init <app-name> --platform node.js --region <region>

# Environments
eb create <env-name> --envvars KEY=value
eb deploy <env-name>
eb status
eb health
eb list

# Logs & SSH
eb logs
eb ssh --setup # pick a keypair whose PRIVATE key exists locally
eb ssh

# Cleanup
eb terminate <env-name> # must type exact env name to confirm

# Blue/green zero-downtime swap
eb create <env-name>-green --cname-prefix <prefix>
eb swap-environment-cnames \
--source-environment-name <env-name> \
--destination-environment-name <env-name>-green
```

8. Exam-Ready Takeaways & High-Yield Notes

- **.ebextensions files execute in alphabetical order** — naming convention (01-, 02-) directly controls execution sequence during deployment.
- **container_commands vs. platform hooks:** container_commands run during the deploy lifecycle before the app is fully live; .platform/hooks/ (prebuild, predeploy, postdeploy) is the modern Amazon Linux 2023 mechanism and gives more granular control over hook timing.
- **Amplify's branch-based CI/CD is native** — it does not require CodePipeline. Each connected branch can have its own build spec, environment variables, and custom domain, deployed independently.
- **EB environment variables set via .ebextensions or the console are stored in plaintext.** For secrets (API keys, DB passwords), the exam expects you to know to reference AWS Secrets Manager or SSM Parameter Store at runtime instead.
- **EB CLI deploys are git-based by default** — uncommitted files will not be included, and if no committed source exists, EB silently falls back to deploying the built-in Sample Application rather than failing loudly.
- **Environment health (Green/Red) measures version-match and instance stability** — not application-level correctness. Always verify Deployed Version separately from health status.
- **Deployment policy choice is a cost-vs-availability tradeoff** that shows up constantly in scenario questions: All-at-once (cheap, has downtime) vs. Immutable (zero downtime, temporarily doubles compute cost) vs. Rolling (partial capacity during deploy) vs. Canary/Traffic-splitting (safest gradual rollout).
- **EB environments are region-scoped resources** and CLI context (.elasticbeanstalk/config.yml) is local, per-directory state that can silently drift from what is actually running in AWS.

9. Recommended Next Steps for This Project

- Retrieve the full eb-engine.log via the console's Request Logs -> Full Logs flow and pinpoint the exact command/line causing 'Engine execution has encountered an error'.
- Once staging is stable, replicate the exact same steps for the myapp-prod environment.
- Connect the front-end/ directory to AWS Amplify with main -> production and dev -> staging branch mappings, setting branch-specific environment variables pointing to each EB environment's CNAME.
- Practice a deliberate blue/green deployment using eb swap-environment-cnames to internalize zero-downtime deployment mechanics for the exam.
- Add IAM least-privilege scoping to the EB instance profile (currently likely using a broad policy) as a follow-on security exercise mapped to Domain 2 of the exam.
- Move any future secrets (API keys, DB credentials) out of .ebextensions and into AWS Secrets Manager or SSM Parameter Store, referencing them at application startup.

This report reflects a genuine, unresolved-at-time-of-writing production debugging session. The value of this project for exam preparation lies not in a clean success story, but in the disciplined, systematic elimination process demonstrated above — exactly the diagnostic reasoning DVA-C02 scenario questions are designed to test.